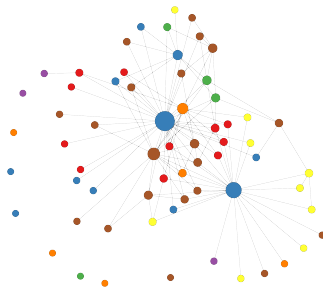


Title of Presentation

Topics



March, 2026

2 Examples

2 Examples

Others

2 Examples

Others

Enumerate list

- 1 Python
- 2 Java
- 3 C++
- 4 C
- 5 Go

List of list (2)

- Word embeddings
 - Word2vec
 - **Skip-Gram**
 - Continuous Bag of Words (CBOW)
- Graph embeddings
 - Shallow embeddings
 - DeepWalk
 - **Node2vec**
 - Struc2vec
 - Structural Deep Network Embedding (SDNE)
 - Metapath2vec
 - Graph Neural Networks (GNN)
 - Graph Convolutional Networks (GCN)
 - Graph Attention Networks (GAT)
 - Graph Isomorphism Networks (GIN)
 - GraphSAGE
 - Graph Autoencoders (GAE)

Multicolors list

- Computer vision
- Natural language processing
- Recommender systems
- Program analysis
- Knowledge graphs
- Bioinformatics
- Chemistry
- Biology
- Physic
- Anomaly detection
- Urban intelligence
- Financial transaction

Description list

Input: Graph data

Output: Community labels

Method: Leiden algorithm

Sequential list display

- Many people use $\text{\LaTeX}$, and many universities have their own Beamer themes

Sequential list display

- Many people use $\text{\LaTeX}$, and many universities have their own Beamer themes
- For Chinese support, use $\text{Xe}\text{\LaTeX}$ as the compiler

List (dynamic) I

- This template is modified from a Tsinghua University project
- Original template: `https://www.overleaf.com/latex/templates/thu-beamer-theme/vwnqzmzndvwyb`
- Minor modifications were made
- Style adjusted to personal preference
- Added small Beamer usage examples
- This is *emphasized text*
- Replaced default fonts with more readable ones

List (dynamic) II

- Fixed inconsistent Chinese font issues
- Changed math fonts to serif for better appearance
- **Use `\alert` to emphasize text**
- This template is modified from a Tsinghua University project
- Original template: <https://www.overleaf.com/latex/templates/thu-beamer-theme/vwnqzmzndvwyb>

Simple equations

$$V = \frac{4}{3}\pi r^3$$

$$V = \frac{4}{3}\pi r^3$$

Equations

$$E = mc^2$$

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G_t] \quad (1)$$

$$Q_{\text{target}} = r + \gamma Q^{\pi}(s', \pi_{\theta}(s') + \epsilon) \quad (2)$$

$$\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c) \quad (3)$$

Equation with numbers

$$\begin{aligned}
 A = \lim_{n \rightarrow \infty} \Delta x & \left(a^2 + \left(a^2 + 2a\Delta x + (\Delta x)^2 \right) \right. \\
 & + \left(a^2 + 2 \cdot 2a\Delta x + 2^2 (\Delta x)^2 \right) \\
 & + \left(a^2 + 2 \cdot 3a\Delta x + 3^2 (\Delta x)^2 \right) \\
 & + \dots \\
 & \left. + \left(a^2 + 2 \cdot (n-1)a\Delta x + (n-1)^2 (\Delta x)^2 \right) \right) \\
 & = \frac{1}{3} (b^3 - a^3) \quad (4)
 \end{aligned}$$

Algorithms

Algorithm 1: My algorithm.

Input : G, s

▷ Graph G and a source node s

Output: All nodes reachable from s

```

1   $Q \leftarrow \emptyset$ 
2   $V \leftarrow \emptyset$ 
3  enqueue( $Q, s$ )
4  append( $V, s$ )
5  while  $Q \neq \emptyset$  do
6       $v \leftarrow \textit{dequeue}(Q)$ 
7      print( $v$ )
8      foreach  $u \in N(v)$  do
9          if  $u \notin V$  then
10             append( $V, u$ )
11             enqueue( $Q, u$ )

```

▷ Create an empty queue

▷ Create an empty set of visited

① Components

Lists

Maths

Tables

Figures

Others

② Examples

Table

Table 1: My table.

Representation	Complexity
Adjacency matrix	$O(V ^2)$
Adjacency list	$O(V + E)$
Edge list	$O(E)$
Incidence matrix	$O(V E)$

Table (custom)

Table 2: My table.

Representation	Advantages	Disadvantages
Adjacency matrix	Simple structure, constant-time edge lookup.	High memory usage for large sparse graphs.
Adjacency list	Memory efficient for sparse graphs, easy to iterate over neighbors.	Edge existence check may be slower.
Edge list	Very simple representation, useful for algorithms that process edges sequentially.	Inefficient for neighbor queries.
Incidence matrix	Useful in some theoretical formulations and algebraic graph analysis.	High memory usage and rarely used in practical implementations.

Table (custom)

Table 3: My table.

IA	Tasks
AI learns patterns	AI automates tasks
AI processes data	AI supports decisions
AI powers assistants	AI transforms industries
AI evolves	AI predicts

1 Components

Lists

Maths

Tables

Figures

Others

2 Examples

Figure

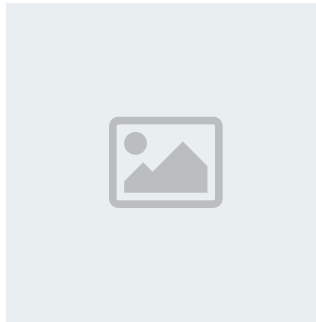
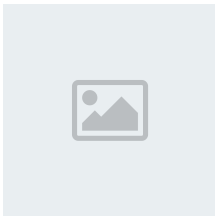


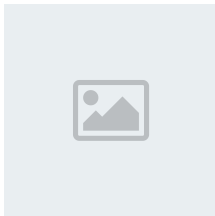
Figure 1: My image.

Source: Author, 2024.

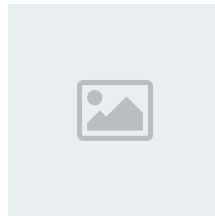
Multiple figures 1



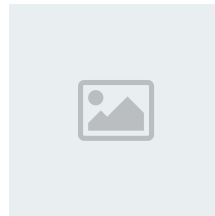
(a) My image.



(b) My image.



(c) My image.

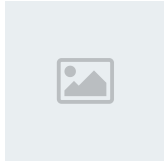


(d) My image.

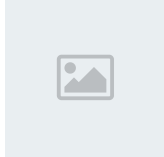
Figure 2: My image.

Source: Author, 2024.

Multiple figures 2



(a) My image.
Source: Author, 2024.



(b) My image.
Source: Author, 2024.

(c) My image.
Source: Author, 2024.

Figure 3: My image.
Source: Author, 2024.

Figure (dynamic)

Figure 4: My image.

Source: Author, 2024.

1 Components

Lists

Maths

Tables

Figures

Others

2 Examples

Blocks 1

Info

General content

Warning

Important message

Example

Example content

Definition

Example definition

Example

Example content

Blocks 2

Theorem (Pythagoras)

$$a^2 + b^2 = c^2$$

Corollary

$$x + y = y + x$$

Proof.

$$\omega + \phi = \epsilon$$



Frame (two columns)

Se enfoca solo en:

- Números (notas)
- Operaciones matemáticas

Se ignora:

- Nombres de cursos
- Detalles del estudiante

Frame (two columns (list of list))

- **Graph traversal**

- Breadth-First Search
- Depth-First Search

- **Shortest path algorithms**

- Dijkstra
- Floyd–Warshall
- Bellman-Ford

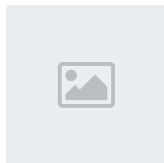
2 Examples

Example 3

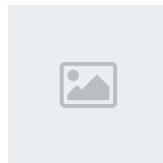
Adjacency matrix

Adjacency matrix A , is an $n \times n$ matrix where:

$$A_{i,j} = \begin{cases} 1 & \text{if there is an edge from vertex } i \text{ to vertex } j \\ 0 & \text{otherwise} \end{cases}$$



(a) My image.



(b) My image.

Figure 6: My image.

Source: Author, 2024.

Example 5

BFS is a graph traversal algorithm that explores nodes level by level, visiting all nodes at a given distance from the starting node before moving to the next level.

Description:

- Uses a queue (FIFO) to manage the nodes to be explored.
- Marks visited nodes to avoid cycles or repeated visits.
- Ideal for finding the shortest path in unweighted graphs.

Example 6

In this lab, students will apply basic programming concepts to solve problems, develop coding skills, and practice writing and debugging efficient solutions.

- Requirements
 - Python
 - Numpy
 - Pandas
 - NetworkX
 - Matplotlib
- Topics
 - Create graphs
 - Graphs visualization
- Lab
 - [Open in Google Colab](#)

Example 7

This section introduces key equations to model and analyze the problem.

Main equation

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$$

Supplementary equations

$$Q_{\text{target}} = r + \gamma Q^\pi(s', \pi_\theta(s')) + \epsilon \quad (8)$$

$$\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c) \quad (9)$$

Example 9

The following table summarizes the key information and results, providing a clear overview of the main variables and their relationships.

Table 4: My table.

Representation	Complexity
Adjacency matrix	$O(V ^2)$
Adjacency list	$O(V + E)$
Edge list	$O(E)$
Incidence matrix	$O(V E)$

Example 10

The following table summarizes the key information and results, providing a clear overview of the main variables and their relationships.

Table 5: My table.

Representation	Complexity
Adjacency matrix	$O(V ^2)$
Adjacency list	$O(V + E)$
Edge list	$O(E)$
Incidence matrix	$O(V E)$

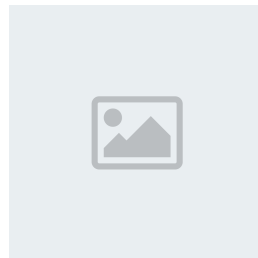


Figure 9: My image.

Source: Author, 2024.

Example 12

Degree

Given a graph $G = (V, E)$.

The degree of a node u , denoted as $\deg(u)$, is defined as the number of edges that are adjacent to this node u .

For undirected graphs:

For directed graphs:

$$\deg(u) = |\{v \in V : (u, v) \in E\}| \quad (11)$$

$$deg(u) = deg^{in}(u) + deg^{out}(u) \quad (12)$$

Where, $deg^{in}(u)$: in-degree and $deg^{out}(u)$: out-degree.

Applications:

- Social networks: users with many connections.
- Biology networks: metabolites involved in many reactions.
- Web: most linked pages.

Example 13

Degree centrality

Given a graph $G = (V, E)$.

Degree centrality considers the node with the highest degree as the most central node in the network G . Degree centrality focuses on individual nodes; it simply counts the number of edges that a node has.

$$DC(v) = \frac{deg(v)}{|V| - 1} \quad (13)$$

Applications:

- Social networks: who has the most friends?
- Internet: which routers or servers handle the most connections?
- Citation networks: which papers are most cited?

Example 14



Figure 11: My image.

Source: Author, 2024.

Example 15

Descomposición

Consiste en **dividir** el problema en sub problemas o en partes más pequeñas (unidades lógicas de menor complejidad); luego, solucionar cada sub problema independientemente. Finalmente, las soluciones a los subproblemas se integran para resolver el problema original completo.

Ejemplo: Sistema para el cálculo de promedio de notas.

Dividir el problema en:

- 1 Ingresar notas
- 2 Calcular suma
- 3 Calcular promedio
- 4 Mostrar resultado

Example 16

Abstracción

Consiste en la capacidad de identificar los aspectos más importantes de un problema o situación y reducir la complejidad al enfocarse en las partes más relevantes, ignorando los detalles innecesarios. En otras palabras, abstraer implica crear una **representación simplificada** de un problema, modelo o sistema, lo cual facilita su comprensión, análisis y resolución.

Ejemplo: Sistema para el cálculo de promedio de notas.

Se enfoca solo en:

- Números (notas)
- Operaciones matemáticas

Se ignora:

- Nombres de cursos
- Detalles del estudiante

Example 18

Secuenciales

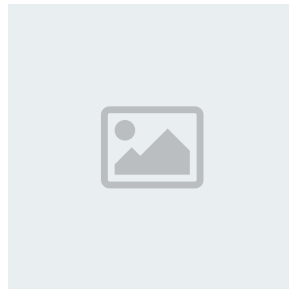
En este tipo de estructura las instrucciones se realizan o se ejecutan una después de la otra.



(a) My image.
Source: Author, 2024.



(b) My image.
Source: Author, 2024.



(c) My image.
Source: Author, 2024.

Figure 12: My image.

Source: Author, 2024.

<https://github.com/win7>

- Erciyes, K. (2018). *Guide to graph algorithms*
- Yao Ma, Jiliang Tang. (2021). *Deep Learning on Graphs*.
- Stamile C., Marzullo A., Deusebio E. (2021). *Graph Machine Learning*.
- Wu, L., Cui, P., Pei, J., Zhao, L., & Guo, X. (2022). *Graph neural networks: foundation, frontiers and applications*.
- Knickerbocker, D. (2023). *Network Science with Python*.
- Labonne, M. (2023). *Hands-On Graph Neural Networks Using Python*.
- Broadwater, K., & Stillman, N. (2025). *Graph neural networks in action*.